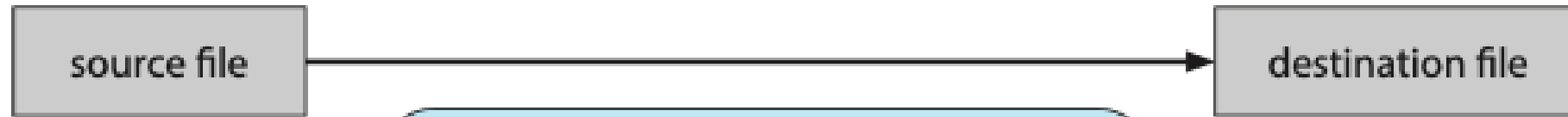

Module 2

Operating System Principles

System Calls

- ❑ Provide an interface to the services made available by an operating system.
- ❑ These calls are generally available as functions written in C and C++.
- ❑ It is a way for a user program to interface with the operating system.
- ❑ The program requests several services, and the OS responds by invoking a series of system calls to satisfy the request.
- ❑ A system call is a method for a computer program to request a service from the kernel of the operating system on which it is running.
- ❑ It acts as a link between the operating system and a process, allowing user-level programs to request operating system services.
- ❑ When a computer software needs to access the operating system's kernel, it makes a system call. The system call uses an API to expose the operating system's services to user programs. **It is the only method to access the kernel system.**

System Calls - Example



Example System-Call Sequence

- Acquire input file name
 - Write prompt to screen
 - Accept input
- Acquire output file name
 - Write prompt to screen
 - Accept input
- Open the input file
 - if file doesn't exist, abort
- Create output file
 - if file exists, abort
- Loop
 - Read from input file
 - Write to output file
- Until read fails
- Close output file
- Write completion message to screen
- Terminate normally

System Calls – Application Programming Interface

- ❑ Application developers design programs according to an application programming interface (API).
- ❑ The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.
- ❑ An API defines the correct way for a developer to request services from an operating system (OS) or other application and expose data within different contexts and across multiple channels.
- ❑ System calls offer the services of the operating system to the user programs via API (Application Programming Interface).

System Calls – Application Programming Interface

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

ssize_t	read	(int fd, void *buf, size_t count)
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

System Calls – Application Programming Interface

❑ Benefits of API:

- ❑ An application programmer designing a program using an API can expect his/her program to compile and run on any system that supports the same API .
- ❑ Actual system calls can often be more detailed and difficult to work with than the API available to an application programmer.

API – System Call – OS Relationship

- ❑ The handling of a user application invoking the `open()` system call.

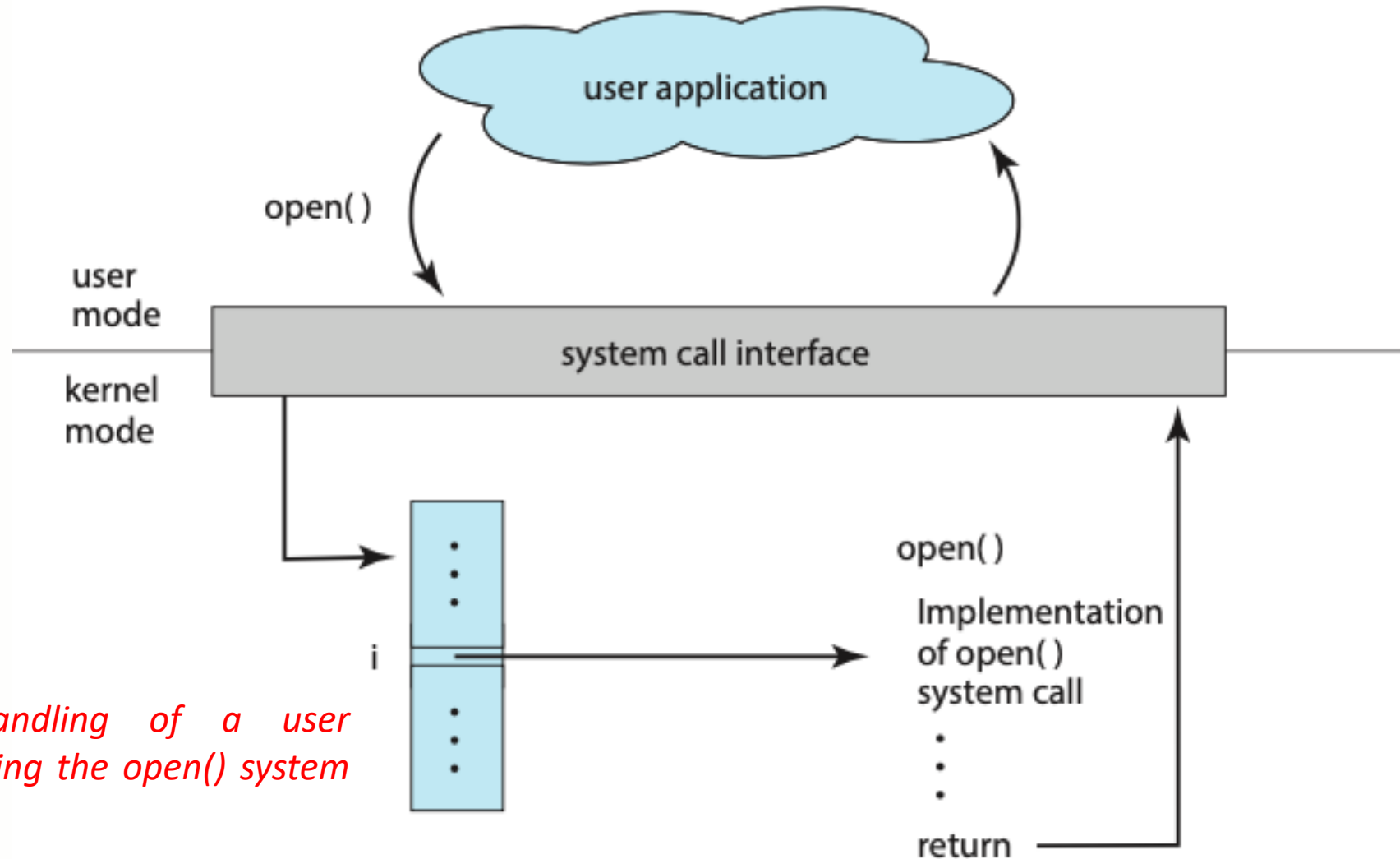


Figure: The handling of a user application invoking the `open()` system call

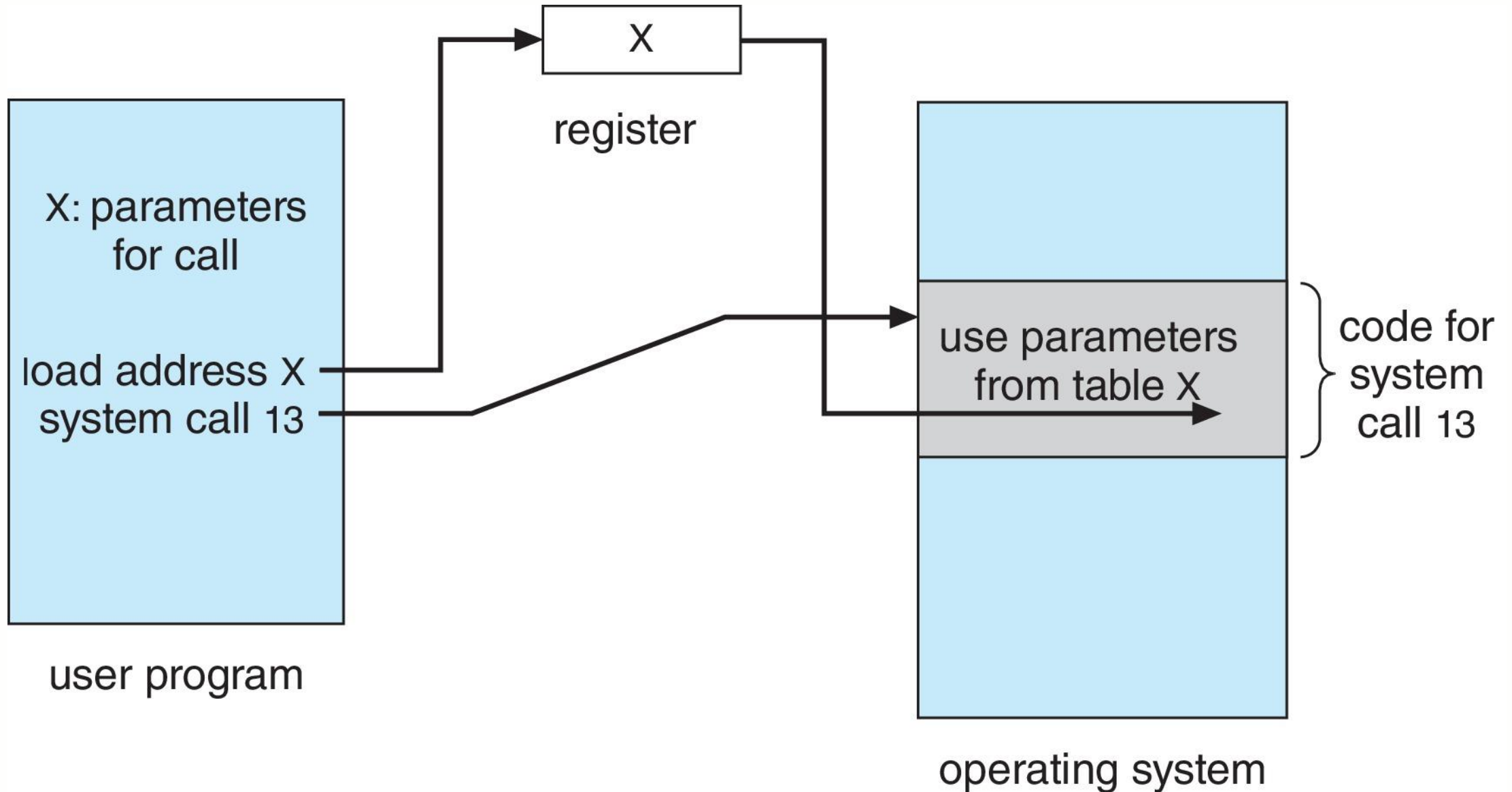
System Call Implementation

- ❑ **Run-time environment (RTE)** is the full suite of software needed to execute applications written in a given programming language.
- ❑ It provides a *system-call interface* that serves as the link to system calls.
- ❑ A number is associated with each system call, and the system-call interface maintains a table indexed according to these numbers.
- ❑ The system-call interface then invokes the intended system call in the operating-system kernel and returns the status of the system call.
- ❑ The caller need know nothing about how the system call is implemented
 - ❑ Just needs to obey API and understand what OS will do as a result call
 - ❑ Most details of OS interface hidden from programmer by API
 - ❑ Managed by run-time support library (set of functions built into libraries included with compiler)

System Call Parameter Passing

- ❑ Often, more information is required than simply identity of desired system call
 - ❑ Exact type and amount of information vary according to OS and call
 - ❑ Three general methods used to pass parameters to the OS
 - ❑ **Simplest:** pass the parameters in registers
 - ❑ In some cases, may be more parameters than registers
 - ❑ **Parameters stored in a block, or table, in memory,** and address of block passed as a parameter in a register. This approach taken by Linux and Solaris
 - ❑ **Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system**
 - ❑ Block and stack methods do not limit the number or length of parameters being passed
- The second and third approaches do not limit the number or length of parameters being passed.*

Parameter Passing via Table/Block



System Calls – working steps

- ❑ A user program wants to read data from a file:
 - ❑ The user program starts executing in the user mode.
 - ❑ The program issues a read system call to request the operating system to read data from the file.
 - ❑ The processor identifies the read system call and invokes it.
 - ❑ The control of the process goes to the kernel mode.
 - ❑ The operating system performs the necessary actions to read data from the file.
 - ❑ Once the read system call's execution gets complete, the control of the process returns to the user mode.
 - ❑ The user program continues executing with the data that was read from the file.

System Calls – Types

System calls can be grouped roughly into six major categories:

❑ Process control

- ❑ create process, terminate process
- ❑ end, abort
- ❑ load, execute
- ❑ get process **attributes**, set process attributes
- ❑ **wait** for time
- ❑ wait event, signal event
- ❑ allocate and free memory
- ❑ **Dump memory** if error
- ❑ Debugger for determining bugs, single step execution
- ❑ **Locks** for managing access to shared data between processes

System Calls – Types

❑ File management

- ❑ create file, delete file
- ❑ open, close file
- ❑ read, write, reposition
- ❑ get and set file **attributes**

❑ Device management

- ❑ Request device, release device
- ❑ Read, write, reposition
- ❑ Get device attributes, set device attributes
- ❑ Logically attach or detach devices

System Calls – Types

☐ Information maintenance

- ☐ get time or date, set time or date
- ☐ get system data, set system data
- ☐ get and set process, file, or device attributes

☐ Communications

- ☐ create, delete communication connection
- ☐ send, receive messages if message passing model to host name or process name
 - ☐ From client to server
- ☐ Shared-memory model create and gain access to memory regions
- ☐ transfer status information
- ☐ attach and detach remote devices

System Calls – Types

❑ Protection

- ❑ Control access to resources
- ❑ Get and set permissions
- ❑ Allow and deny user access

System Calls – Example

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()